Sādhanā Vol. 41, No. 7, July 2016, pp. 727–753
DOI 10.1007/s12046-016-0509-4
© Indian Academy of Sciences

Comparison between hybridized algorithm of GA–SA and ABC, GA, DE and PSO for vertical-handover in heterogeneous wireless networks

SHIDROKH GOUDARZI[1,*], WAN HASLINA HASSAN[1], MOHAMMAD HOSSEIN ANISI[2] and SEYED AHMAD SOLEYMANI[3]

[1]Communication System and Network (iKohza) Research Group, Malaysia-Japan International Institute of Technology (MJIIT), Universiti Teknologi Malaysia, Jalan Semarak, 54100 Kuala Lumpur, Malaysia
[2]Department of Computer System and Technology, Faculty of Computer Science and Information Technology, University of Malaya, 50603 Kuala Lumpur, Malaysia
[3]Department of Computing, Faculty of Computing, University of Teknologi Malaysia, Johor Bahru, Malaysia
e-mail: shidrokhgoudarzi@gmail.com

MS received 18 May 2015; revised 4 October 2015; accepted 8 March 2016

Abstract. Genetic algorithms (GAs) and simulated annealing (SA) have emerged as leading methods for search and optimization problems in heterogeneous wireless networks. In this paradigm, various access technologies need to be interconnected; thus, vertical handovers are necessary for seamless mobility. In this paper, the hybrid algorithm for real-time vertical handover using different objective functions has been presented to find the optimal network to connect with a good quality of service in accordance with the user's preferences. As it is, the characteristics of the current mobile devices recommend using fast and efficient algorithms to provide solutions near to real-time. These constraints have moved us to develop intelligent algorithms that avoid slow and massive computations. This was to, specifically, solve two major problems in GA optimization, i.e. premature convergence and slow convergence rate, and the facilitation of simulated annealing in the merging populations phase of the search. The hybrid algorithm was expected to improve on the pure GA in two ways, i.e., improved solutions for a given number of evaluations, and more stability over many runs. This paper compares the formulation and results of four recent optimization algorithms: artificial bee colony (ABC), genetic algorithm (GA), differential evolution (DE), and particle swarm optimization (PSO). Moreover, a cost function is used to sustain the desired QoS during the transition between networks, which is measured in terms of the bandwidth, BER, ABR, SNR, and monetary cost. Simulation results indicated that choosing the SA rules would minimize the cost function and the GA–SA algorithm could decrease the number of unnecessary handovers, and thereby prevent the 'Ping-Pong' effect.

Keywords. Vertical handovers; genetic algorithms (GAs); simulated annealing (SA); heterogeneous wireless networks.

1. Introduction

In recent decades, we have witnessed astonishing developments in wireless applications, devices, and networks. In heterogeneous wireless networks, the vertical handover (VH) is an important factor in the provision of seamless mobility between varied network environments and an essential feature of all next-generation all-IP mobile network endeavours. Since no single access technology can offer ubiquitous coverage and continuously high quality of service (QoS), multimode mobile terminals will have to roam between the various accesses technologies to retain network connectivity and user satisfaction. These heterogeneous wireless access networks classically vary in terms of signal strength, coverage, data rate, latency, and loss rate. VH occurs when a mobile terminal decides to switch between networks. The VH process comprises three main functions: system discovery, handover decision, and handover execution. The VH decision is important for providing a low cost, highly available network environment that can achieve the desired quality of service (QoS) or quality of experience (QoE). During the handover decision stage, the mobile device decides to which network it should connect.

*For correspondence

Many challenges are present in the VH decision phase during the handover process. One of the challenging problems during vertical handover is the selection of an optimal network that maximizes the end users' satisfaction. On the one hand, sometimes the terminal is moving rapidly along its route, so the algorithms that provide the VH decision phase must also be fast and able to give a solution near to real-time in such dynamic scenarios (the mobility aspect is a key driver for future Internet, within the field of mobility and ubiquitous access to networks). On the other hand, some decision algorithms handle many parameters that involve quite a lot of floating-point arithmetic calculations, and the computational effort increases with the required precision for the solutions, the number of QoS parameters or the available networks discovered during the movement of the terminal. A high computational effort is in conflict with the low response time restriction, especially taking into account, the low-performance processors embedded in many mobile devices.

Hence, there is a critical need to develop effective vertical handover decision algorithms (VHDAs) that not only cultivate a hybrid VHDA that utilizes some form of intelligence for decision-making but is also able to dynamically adapt to varying conditions in a timely manner given the rapid change in the wireless environment. Some VHDAs including those involving computational intelligence techniques have been proposed in current research literature. According to Chandralekha and Behera [1], the decision problem is formulated as a group of multiple objective optimization problems and simulated using genetic algorithms. The simulation result showed that the number of handovers could be minimized if optimized network parameter values were taken into consideration.

Another category uses hybrid techniques where the algorithm was inspired by the Nash equilibrium [2], known from the game theory. This study tried to seek a Pareto optimum solution under the Nash equilibrium of user utility and network provider utility, and used an elitist selection and individual migration of multi-objective genetic algorithm to improve the decision efficiency. Jaraiz-Simon *et al* [3] proposed a new model to develop intelligent algorithms that avoided the slow and massive computations associated with direct search techniques, hence reducing the computation time. The authors developed an algorithm based on genetic algorithms (GAs) in order to obtain a better performance than SEFISA and overcame the limitations exposed before, mainly the limitation of the number of generations, the appearance of compulsory stop criterion, the overflow of search space limits, stagnation of optimal solutions along generations, and so on.

According to Lin *et al* [4], the hybridization of genetic algorithms with existing algorithms showed better performance when compared to either the GAs or the existing algorithms used alone [5]. As reported by Giupponi *et al* [6] and Wilson *et al* [7], there were several existing multicriteria-based algorithms with the aid of artificial intelligence tools such as fuzzy logic, neural networks, and genetic algorithms that suffered from scalability and modularity problems. They were unable to cope easily with the increased numbers of RATs and criteria in the heterogeneous wireless networks (HWN). These algorithms took all the inputs from the different RATs at once to one fuzzy logic block; hence, they suffered from scalability and complexity problems when more RATs or membership functions were added due to the exponential increase on the number of inference rules (Giupponi *et al* [6, 7].

In the context of future wireless networks, many analyses, studies, and tutorials have been proposed in the literature [8–11]. These algorithms were classified into different groups based on the decision technique expended. Rakovic and Gavrilovska [12] proposed a novel method for radio access technology (RAT) selection, namely, the Hopfield neural network RAT selection mechanism (HRM), that utilized the Hopfield neural networks as a strong decision making tool. A new approach using information about data rate, monetary cost, and received signal strength as different parameters to make a handover decision has been reported by Çalhan and Çeken [13]. The main weaknesses were exactly linked to the computation of the error function and Jacobian inversion for acquiring a matrix in which the dimensions were equal to the total of all the weights in the neural network. Hence, the requirement for memory was very high [14, 15]. Existing algorithms [13] considered service fee, received signal strength information (RSSI), user preference, etc. The proposed algorithm compared to the traditional RSSI based algorithm, enhanced outcomes significantly for both user and network as a consequence of the proposed fuzzy based handover systems [16]. In terms of hybrid categories, Nan *et al* [17] proposed a PSO–FNN-based VHDA that could make a reasonable handover decision intelligently based on the study of network status. Liu and Jiang [18] reported a novel VHDA built on fuzzy logic with the assistance of the grey theory and the dynamic weights adaptation. A neuro-fuzzy multi-parameter-based VHDA was proposed by Singhrova and Prakash [19] where the results of performance evaluation, carried out by a handover quality indicator (used to quantify QoS) which is related to the 'Ping-Pong' effect, ESA and throughput, proved that the proposed VHDA offered better QoS than the existing vertical handover methods. Pahlavan *et al* [20] was a good representation of the application of a fuzzy logic-based normalized quantitative decision algorithm and a differential prediction algorithm that had good accuracy.

Although the vertical handover strategies mentioned above had their own advantages, they did not consider the

complexity conditions of the network selection, and the allocation of lower computation cost function was also unreasonable. Clearly, the decision process needed to rely on a fast intelligent algorithm to perform an accurate decision, and to switch to the best candidate network within a very short period of time.

Hence, the objective of this study was to propose a novel, hybrid network selection optimization algorithm that took advantage of two existing optimization methods to meet the above stated requirements. It was expected to provide more reliable results than a single run of the GA. The combination of SA rules with GA has three purposes: (1) to serve as a validation algorithm for the outcome of the GA, (2) to detect calculation results that are unacceptable, i.e. higher than the global optimum, and (3) to merge new generations of crossover and mutation steps. In this algorithm, in order to create new populations, the rules of SA were used. The proposed method is a step towards future computer-based optimization methods where huge uncertainties by the optimization algorithm need to be avoided. To do this, a genetic algorithm was combined with the simulated annealing (SA) optimization. The proposed approach synthesizes the capabilities of both search methods to avoid the drawbacks of GA. The combined algorithm differed from the already introduced evolutionary search methods in which it used a modified, simulated annealing optimization method to validate the outcome of the genetic algorithm. The proposed GA-SA-based VHDA could make reasonable handover decisions intelligently according to the study of the network status. The cost function was also expected to be minimized by choosing the SA rules. In addition, this algorithm is expected to reduce the number of unnecessary handovers, and hence, avoid the 'Ping-Pong' effect. The outline of this paper is as follows: first, the optimization problem is described in section 2. Then, the methodology of optimization algorithms and comparison of optimization algorithms is explained based on recent optimization algorithms: artificial bee colony (ABC), genetic algorithm (GA), differential evolution (DE), and particle swarm optimization (PSO) in section 3. The system model is described in section 4. Section 5 provides the calculations and discussion of the results. Finally, section 6 summarizes the conclusions.

2. The optimization problem

The efficient adjusting of the QoS weights to determine the best network among available ones is very significant for wireless networks. In order to find the best network, we need to know the merit of each available network. For this purpose, we need to design a function or metric to acquire the merit of this network. Firstly, a set of weights assigned to each one of the QoS parameters is used to measure the quality of the network, and they are based on the user's preferences and network characteristics. A general profile that includes any QoS parameter can have any weight assigned between 0 and 1. This measure is given by a function, which is called cost function. This function is evaluated in the VH decision-making stage. Thus, the optimization problem consists of searching the best solution that, when applied to each network, returns the lowest cost to the network, which will then be chosen as the best solution for the VH decision stage. The GA component assigns a suitable weight (w_1, w_2,…, w_i) for each initial decision according to the objective function that is specified by the operator according to the importance and sensitivities of the access network selection criteria to the different characteristics of a wireless heterogeneous environment.

2.1 *Cost function*

The vertical handover cost function is a measurement of the cost utilized by handing off to a particular network. It is evaluated for each network that covers the service area of a user. The network choice that results in the lowest calculated value of the cost function is the network that would provide the most benefits to the user. As is known, different types of services require various combinations of reliability, latency, and data rate. Thus, we consider service type as the main metric. Moreover, monetary cost is of major consideration to users, as different networks may employ different billing strategies that may affect the user's choice to handover. In terms of network conditions, network-related parameters such as available bandwidth and network latency may need to be considered for effective network usage. Use of network information in the choice to handover can also be useful for load balancing across different networks. In addition, a variety of parameters can be employed in the handover decision to guarantee the system performance, for example, the bit error rate (BER). Mobile terminal conditions also include dynamic factors such as velocity, moving pattern, moving histories, and location information.

For any of the service types, two sets of relative priorities are defined. These two sets are interface priorities and application priorities where relative priorities among available interfaces in a device are interface priorities [21] and relative priorities among five types of services are application priorities. Based on the arrangement of the literal priority, scores between 1 and 9 are assigned automatically at the backend, where 9 denotes the most preferred and 1 denotes the least preferred priority. Setting priorities among available interfaces in the multimode terminal should be an important parameter from the user side because, for example, most users may like to give the



Figure 1. Interface priorities and application priorities [41–44].

cellular interface the highest priority especially for voice applications due to its almost ubiquitous coverage. QoS is another important parameter in order to ensure user satisfaction to the fullest extent. The user can establish preferences for the different QoS parameters depending on the service required. The experimentation is more realistic considering different sets for the user's preferences, where the QoS parameters that are more important have higher values of their weights. The whole process is illustrated in figure 1.

At this stage, suitable limit values (upper and lower) for the five QoS parameters are mapped at the backend for each of the five service types. While fixing the limit values, it is important to note that high values are not always better for all the five QoS parameters. It is always preferable to have values as high as possible for bandwidth, but as low as possible for delay and BER. These values are based on the contexts such as QoS requirements of specific service types and interface capabilities.

At the next stage, ranking of the available networks is performed based on the interface priorities scores and application priorities scores assigned at stage 1. Consider a set of candidate networks $S = \{s_1,\ldots, s_N\}$ and a set of quality of service factors $Q = \{q_1,\ldots, q_M\}$ where N is the number of candidate networks and M is the number of quality of service factors. In addition, we consider that each QoS factor has its own weight and this weight shows the effect of the factor on the network or user. Thus, cost function for each network can be calculated using Eq. (1) where W_N is calculated by the analytic hierarchy process (AHP) [22]. This process is chosen due to its ability to vary its weighting between each factor based on network conditions and user preferences.

$$C_N = W_{\text{Interface}} \times \sum_{j=1}^{M} q_j \times W_j. \tag{1}$$

With the above definitions, the AHP method is described as follows.

Subsequently, the relative scores among the QoS scores set are calculated. Relative scores between any two particular scores are calculated using (2) where $R_{q_iq_j}$ is the relative score between parameters q_i and q_j, and S_{q_i} and S_{q_j} are their respective scores.

$$\begin{cases} R_{q_iq_j} = \left(1 - \dfrac{S_{q_i}}{S_{q_j}}\right) \times 10 & j > i \\ R_{q_jq_i} = \dfrac{1}{R_{q_iq_j}} & j < i \\ R_{q_iq_j} = 1 & i = j \end{cases} \tag{2}$$

$X = \{X_{ij}\}$ is $M \times M$ matrix, which X_{ij} represents the priority scores of each factor and is initialized as follows (3):

$$X = \begin{bmatrix} 1 & R_{q1q2} & R_{q1q3} & R_{q1q4} & R_{q1q5} \\ \dfrac{1}{R_{q1q2}} & 1 & R_{q2q3} & R_{q2q4} & R_{q2q5} \\ \dfrac{1}{R_{q1q3}} & \dfrac{1}{R_{q2q3}} & 1 & R_{q3q4} & R_{q3q5} \\ \dfrac{1}{R_{q1q4}} & \dfrac{1}{R_{q2q4}} & \dfrac{1}{R_{q3q4}} & 1 & R_{q4q5} \\ \dfrac{1}{R_{q1q5}} & \dfrac{1}{R_{q2q5}} & \dfrac{1}{R_{q3q5}} & \dfrac{1}{R_{q4q5}} & 1 \end{bmatrix}. \tag{3}$$

Then, when each element of the matrix X is divided with the sum of its column (4), the normalized relative weight is obtained.

$$X_{ij} = \frac{X_{ij}}{\sum_{i=1}^{M} X_{ij}} \tag{4}$$

The normalized matrix X is called w_{norm} which is shown in (5).

$$w_{\text{norm}} = \begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} & w_{15} \\ w_{21} & w_{22} & w_{23} & w_{24} & w_{25} \\ w_{31} & w_{32} & w_{33} & w_{34} & w_{35} \\ w_{41} & w_{42} & w_{43} & w_{44} & w_{45} \\ w_{51} & w_{52} & w_{53} & w_{54} & w_{55} \end{bmatrix}. \tag{5}$$

Next, the average values of each row are calculated to give the priorities for each factor by (6) which is shown in (7).

$$\bar{w}_i = \frac{w_{i1} + w_{i2} + w_{i3} + w_{i4} + w_{i5}}{5}. \quad (6)$$

The normalized vector (7) is called the priority vector. Since it is normalized, the sum of all the elements in priority vector is 1. The priority vector shows relative weights among them.

$$W_N = \begin{bmatrix} \bar{w}_1 \\ \bar{w}_2 \\ \bar{w}_3 \\ \bar{w}_4 \\ \bar{w}_5 \end{bmatrix}. \quad (7)$$

The set of quality of service is shown by vector Q_N in (8):

$$Q_N = [B \quad E \quad A \quad S \quad C] \quad (8)$$

Which these QoS parameters are as follows: B = bandwidth (Mbps), E = BER (dB), A = available bit rate (ABR) (bps), S = signal to noise ratio (SNR) (dB), and C = monetary cost (eur/MB). At the end, based on Eq. (1), the cost values of each user's requested services from the network can be computed.

3. Methods

In this paper, we consider a heterogeneous wireless environment with the co-existence of WLANs with faster, high-bandwidth, lower-cost, and short-distance access, and the universal mobile telecommunications system (UMTS) with slower, higher-cost, and long-range access networks that are always connected. These algorithms can be programmed with small-sized codes, which are needed for embedded applications. It must be taken into consideration that, these days, there is a demand that mobile nodes provide the same performance at a lower price which can lead to reduced power consumption and ultimately improve the performance and mobility. We have provided a brief introduction for each algorithm; then, the operation is explained with a pseudo code (Appendices 1–5) to facilitate the implementation and the use of such algorithms by researchers and practitioners, and figures as a visual aid. Finally, we indicate that our final scheme to solve the vertical handover optimization problem relies on the proposed algorithm, due to its better performance, which will be shown in the later sections.

3.1 *Artificial bee colony (ABC)*

ABC [23] simulates the intelligent foraging behavior of a honeybee swarm. In the ABC algorithm, each cycle contains three steps: At the beginning, a set of food source is randomly selected by bees and their amounts are given. A new food source is produced, and the new nectar amount can be obtained by the new food source. The artificial bees randomly discover a population of initial solution vectors and then iteratively improve them by employing the strategies: moving towards better solutions by means of a neighbor search mechanism while abandoning poor solutions [24].

3.2 *Genetic algorithm (GA)*

Genetic algorithm (GA) search methods are rooted in the mechanisms of evolution and natural genetics. The interest in heuristic search algorithms with underpinnings in natural and physical processes began as early as the 1970s, when John [25] first proposed genetic algorithms. In the field of artificial intelligence, genetic algorithms have emerged as powerful tools to solve the NP-hard problem. It is very simple to program a GA. GAs are based on an analogy with the genetic structure and behavior of chromosomes within a population of individuals using the following foundations:

- Individuals in a population compete for resources and mates;
- The most successful individuals in each 'competition' will produce more offspring than the individuals that perform poorly;
- Genes from 'good' individuals propagate throughout the population so that two good parents will sometimes produce offspring that are better than either parent; and
- Thus, each successive generation will become more suited to their environment.

Genetic algorithms (GAs) are very popular but they suffer from three main problems [26]. The first problem is mostly related to premature convergence in genetic algorithm optimization, whereby this problem is a result of high reliance on crossover. This can have an effect on the population by making the population more homogeneous and consequently, the search for the best solution is very slow in the mutation step. The second problem of genetic algorithms is related to the convergence of the optimal solution after finding a near-optimal solution. The third problem is the high level of memory usage by genetic algorithms. Since a genetic algorithm must keep a large population of solutions in its memory, this results in increased memory requirements when the problem dimensions increase. The flow chart of the genetic algorithm is presented in figure 2.

In this study, we show the hybridization of genetic algorithms (GAs)–simulating annealing (SA) to improve the performance of a GA for wireless network selection optimization. The idea of this hybrid GSAVHO algorithm is to use the SA rules to improve the efficiency of the GA's mutation and crossover. Section 4 presents the

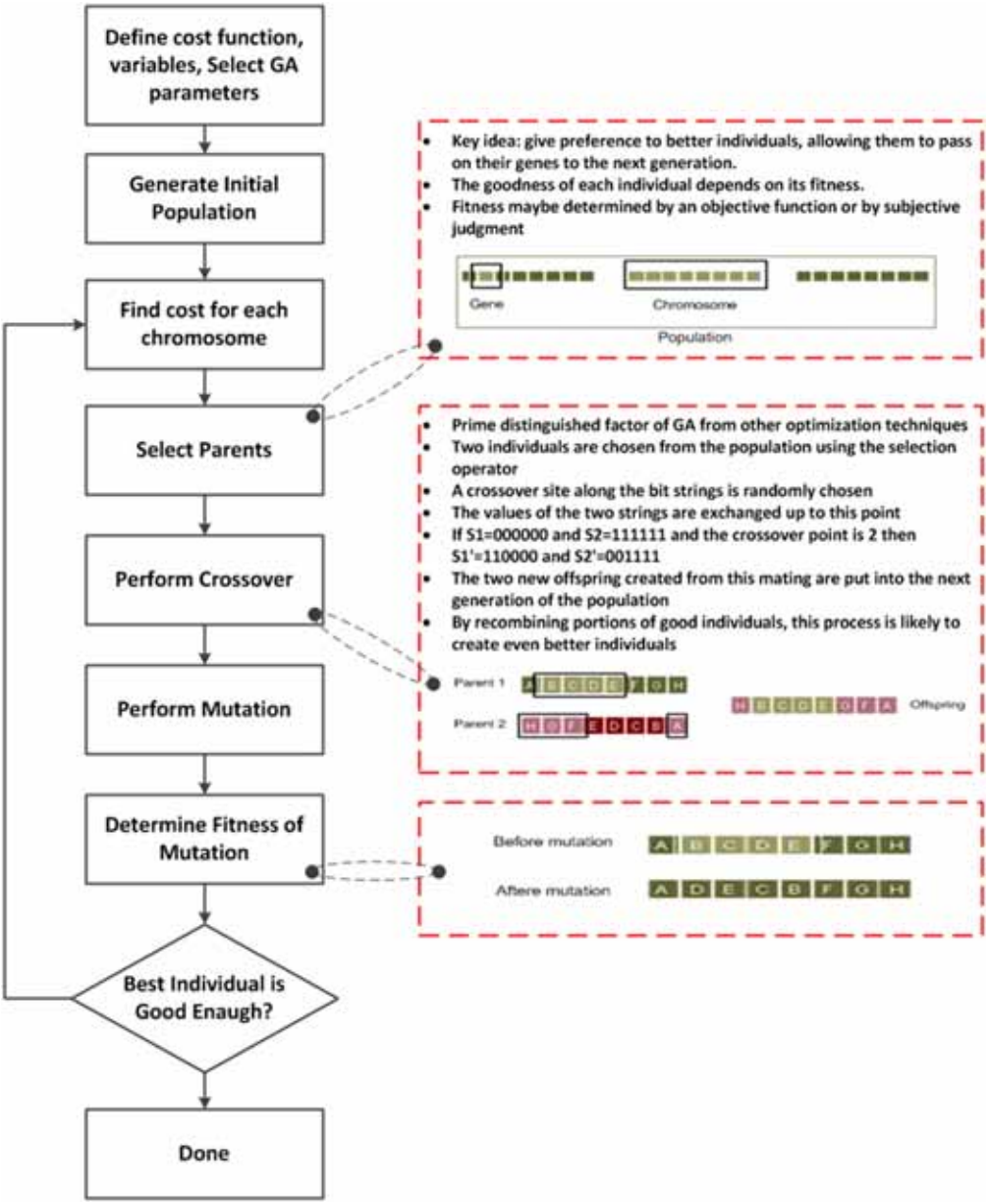


Figure 2. Flow chart of the genetic algorithm (GA).

GSAVHO hybrid algorithm with its detailed algorithmic steps.

3.3 *Differential evolution (DE)*

One of the evolutionary algorithms, which have been presented recently, is the differential evolution (DE) algorithm [27]. In the DE algorithm, a greedy scheme is applied to select one of them for the next generation. The DE algorithm has received significant interest from researchers studying in different research areas on different problems due to the fact that this algorithm has a high level of simplicity and flexibility by using the self-adaptability feature, a crossover operation, and a greedy process for the selection, which can make the DE a fast converging evolutionary algorithm.

3.4 *Particle swarm optimization (PSO)*

PSO is a kind of swarm intelligence, which uses a number of particles that constitute as a swarm, moving around in the search space looking for the best solution. Particles are treated as a point in an N-dimensional space, which modifies their "flying" base on their own flying experience as

well as the flying experience of other particles. Particles can change their position's based on various types of information, namely current positions, current velocities, the distance between the current position and personal best position (pbest), and the distance between the current position and global best position (g best). Particles can track their coordinates in the solution space, which are associated with the best solution (fitness) that has been realized thus far by these particles [28].

3.5 *Simulated annealing*

Annealing is the cooling process all through in which a low energy state is achieved in a solid. In the annealing method, the solid is first heated and melted at a high temperature, in which position all particles move at random. The solid is then slowly cooled until the particles place themselves in their lowest energy state, which is the ground condition. To avoid the resulting crystals from showing defects or even lacking all crystalline order, the cooling schedule should be sufficiently slow [29]. In the simulated annealing technique, a new solution is created through a neighborhood structure and a generation method. The neighborhood structure states a set of solutions, S_s, which is "close" to the present solution, s, in some sense. The generation method is a means of choosing a new solution from the neighborhood S, of the solutions

3.6 *Hybrid of Genetic and simulated annealing algorithms*

In this work, we have developed an algorithm based on the hybridization of genetic (GAs) and simulating annealing (SA) algorithms in order to obtain better performance and overcome the limitations of GA, which has been outlined in section 3.2. The reasons behind the choice of the GA and SA hybrid are because they have proven to be efficient and strong in search processes, and have produced near optimal solutions, making them suitable for solving large combinatorial optimization problems. Unlike the other algorithms, GAs are biologically inspired algorithms for conducting random search and optimization guided by the principles of natural evolution and genetics. As mentioned before, GA has a strong global search ability in solving the aforementioned problems, but it also has limitations such as a premature and slow convergence rate, local optimum and ignoring cooperation between populations as well as weak local search ability [30]. On the other hand, SA has strong local search ability and no premature problems. Therefore, the hybridization of GA and SA can overcome the limits of each of the two methods, bringing into play their respective advantages, and improve the solving efficiency.

To hybridize GA with SA, we have used SA rules to combine and compare the populations of GA. Figure 3

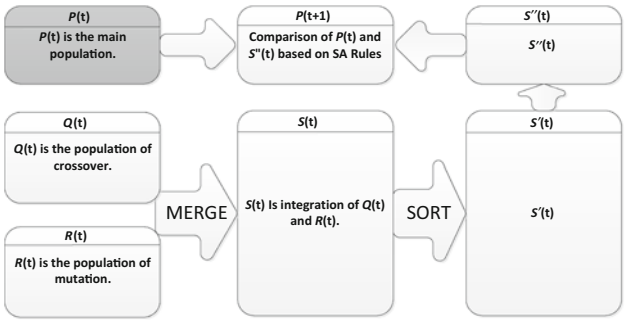


Figure 3. Workflow of (GSAVHO).

shows the workflow of the GA–SA algorithm. In this figure, $P(t)$ is the main population, $Q(t)$ is the population of crossover, and $R(t)$ is the population of mutation. The integration of $Q(t)$ and $R(t)$produces $S(t)$. In the next step, $S(t)$ must be sorted and the sorted population is called $S^{'}(t)$. In $S^{'}(t)$, the top population is the better solution. In $P(t)$, the number of elements is npop (); then, we select the same elements npop () of the top populations of $S^{'}(t)$and label them $S^{''}(t)$. Subsequently, the comparison of $P(t)$ and $S^{''}(t)$ based on SA r rules is needed. At this stage, the population of a new generation has been created.

GAs essentially use three genetic machinists: selection, crossover, and mutation, which simulate the procedure of natural selection and provide them their powerful search ability. Subsequently, the main operators are explained:

Selection: Selection is the process of choosing potentially better individuals to form a mating pool. There are many selection schemes such as Roulette Wheel selection, Boltzmann selection, Rank selection, Tournament selection, and Elitist selection. Elitist selection is expended in this paper and the latter is used to prevent the best solution disappearing in the next generation. The idea of Elitist selection is that some of the best performing individuals are kept and used to replace the same number of the worst individuals in the next generation. That is, the offspring will progress to be at least as good as their parents.

Crossover: The crossover operator simulates the development of gene recombination, which aims to pass the best genes on to the next generation. Usual crossover methods comprise one-point crossover and two-point crossover. According to Tsai *et al* [30], the two-point crossover is greater than the one-point crossover; thus, the two-point crossover is used in this study. The idea of the two-point crossover is that two crossover sites are chosen randomly and then the genes amongst the two sites in the two parent networks are switched.

Mutation: Mutation is an essential genetic operator that raises the variety of the population, even though the probability of mutation is minimal. The idea of the mutation operator is that with a certain probability and in a way that is specific to each individual's every gene, the gene is

exchanged with a new gene not contained in the individual. Notice that the list of the network must be sorted and it should be shown that there are not any duplicate networks in the list after the operation.

Creation of a new generation by SA rules: Based on literature, some strategies have been proposed to merge the population in the GAs. In one strategy called the predefined shares, each set has the same portion and n(pop) is a combination of $P(t) + Q(t) + R(t)$ in which $P(t)$, $Q(t)$, and $R(t)$ are the main population, crossover population, and mutation population, respectively. In addition, in another strategy, called the predefined probabilities, each set has the same probabilities for integration. In this approach, the total probability of three sets is equal to 1. In our strategy, the combination and comparison amongst populations of parents and children are based on simulated annealing rules.

The whole population must be created at a fixed temperature. With this purpose in mind, we have defined that in the external iteration, the temperature should be decreased. In other words, all the processes involved in the creation of a new population including crossover and mutation were moved into the internal iteration, which was named the MaxSubIt () for showing the maximum number of subiterations. We found that the number of parameters increased because the parameters of GA and SA should be defined, for instance, the initial temperature (T_0) and reduction rate of temperature was shown with alpha = 0.99, and were related to SA.

3.7 *Comparison of optimization algorithms*

The performance of the proposed method was compared with well-known optimization algorithms, ABC, GA, DE, and PSO. Thirty trial runs were performed for each method. The performance of the different algorithms was compared using three criteria over 30 different runs: (1) the average of best cost (BC) of the cost function obtained in each trial; (2) the number of function evaluations (N_{fe}), as represented by the number of trials required for the objective function to reach its known target value; (3) the standard deviation of the function values (SD) obtained in all trials; and (4) the computation time (CT) to complete the algorithm. This is particularly relevant in a real-time application when the computer system must respond quickly to some external event.

For calculating the cost function, MATLAB provided sets of functions for measuring absolute cost. Since, the real-time applications need less cost function in addition to high performance, cost function of GSAVHO was examined to assess the efficiency of the algorithm. The results demonstrated the comparison of cost function between our heuristic and other methods. As we can see, GSAVHO dramatically lowered the cost function. In other words, it was evident that the cost function of the proposed GSAVHO was lower by using SA rules in the integration of the new population phase than GA by merging populations after the crossover and mutation phases without SA rules. Table 1 shows the parameter setting of the three algorithms for the comparison between algorithms.

The efficiency is determined by recording the number of function evaluations N_{fe} for each optimization algorithm, where a low value of N_{fe} means a higher efficiency. This paper has used the number of function evaluations (N_{fe}) to measure the performance of algorithms and has defined the optimal model to be the one that consumes the fewest (N_{fe}) to solve the problem. For GSAVHO and other algorithms, the numbers of function evaluations were calculated by Eq. (9) [31]:

$$N_{fe} = 2 \times G_n \times P_n + P_n, \tag{9}$$

where G_n is number of generations where the best solution was obtained and P_n is the number of populations. The standard deviation of the function values of other algorithms gave larger standard deviations of function values than the proposed algorithm, and hence their solutions' qualities were deduced to be less stable. In support, the proposed algorithm gave smaller standard deviation of function values than that given by others; hence, it had a

Table 1. Parameter setting.

ABC		GA		DE		PSO		GSAVHO	
Max iteration	100	Max iteration	100	Max iteration	100	Max iteration	100	Max iteration	100
Npop	3	*Npop*	3	*Npop*	3	*Npop*	3	*Npop*	3
n_o	50%	*pC*	0.7	*CF*	0.8	*w*	0.1	*pC*	2
n_e	50%	*pM*	0.4	*f*	0.5	*C1*	1	*pM*	1
n_s	1	*mu*	0.02			*C2*	2	*mu*	0.1
Limit	$Ne \times D$					*mu*	0.1	T_0	10
								alpha	0.99

Npop, population size; n_o, onlooker number; n_e, employed bee number; n_s, scout number; D, dimension of the problem; *pC*, Crossover percentage; Mutation percentage; *mu*, Mutation Rate; *CF*, crossover factor for DE; *f*, scaling factor; *w*, inertia weight; *C*1, *C*2, lower and upper bounds of the random velocity rule weight; *mu*, mutation rate; T_0, initial temperature; alpha, temp reduction rate

Table 2. The results of ABC, GA, DE, PSO and GSAVHO algorithms based on N_{fe}, number of function evaluation; SD, standard deviation; CT, computation time and BC, best cost.

	ABC				GA				DE			
Run	N_{fe}	SD	CT	BC	N_{fe}	SD	CT	BC	N_{fe}	SD	CT	BC
Run1	31.321	0.018	0.936	0.216	31.349	0.026	0.613	0.205	32.291	0.029	0.566	0.227
Run 2	16.105	0.061	0.472	0.214	16.123	0.073	0.372	0.161	15.944	0.052	0.310	0.202
Run 3	65.479	0.021	1.547	0.227	54.604	0.053	0.918	0.115	63.486	0.006	0.910	0.201
Run 4	46.869	0.014	1.406	0.184	39.755	0.035	0.727	0.161	46.846	0.006	0.704	0.179
Run 5	41.710	0.017	1.085	0.221	41.514	0.010	0.675	0.110	41.722	0.020	0.635	0.215
Run 6	60.616	0.021	1.534	0.196	56.473	0.056	0.872	0.126	61.216	0.035	0.849	0.201
Run 7	18.460	0.002	0.509	0.177	16.697	0.021	0.350	0.085	16.989	0.031	0.301	0.176
Run 8	59.947	1.126	1.453	0.235	51.948	0.044	0.927	0.248	58.877	0.041	0.855	0.262
Run 9	63.764	0.009	1.493	0.204	52.750	0.048	0.919	0.110	63.271	0.004	0.863	0.213
Run 10	43.218	0.019	1.066	0.207	38.873	0.038	0.678	0.106	43.186	0.009	0.635	0.211
Run 11	27.595	0.026	0.867	0.207	23.909	0.136	0.467	0.101	26.933	0.060	0.508	0.198
Run 12	55.443	0.014	1.261	0.199	45.955	0.083	0.900	0.151	53.354	0.035	0.735	0.194
Run 13	21.588	0.023	0.587	0.215	18.274	0.046	0.437	0.122	21.308	8.377	0.370	0.197
Run 14	54.751	0.005	1.299	0.265	46.018	0.068	0.847	0.176	54.499	0.016	0.845	0.218
Run 15	36.267	0.040	1.084	0.216	33.867	0.026	0.561	0.204	36.169	0.031	0.566	0.214
Run 16	21.188	0.060	0.575	0.251	20.535	0.040	0.392	0.172	21.261	0.012	0.385	0.174
Run 17	19.821	0.011	0.575	0.233	19.480	0.024	0.392	0.202	19.939	0.005	0.340	0.225
Run 18	31.738	0.029	0.863	0.199	28.513	0.036	0.542	0.142	31.855	0.077	0.508	0.201
Run 19	17.043	0.085	0.361	0.202	16.512	0.049	0.280	0.133	16.429	0.091	0.241	0.196
Run 20	23.727	0.080	0.416	0.195	18.423	0.177	0.319	0.162	22.482	0.118	0.267	0.169
Run 21	11.752	0.015	0.344	0.227	11.684	0.059	0.260	0.199	11.765	0.017	0.507	0.224
Run 22	54.370	0.012	1.356	0.203	53.913	8.377	0.808	0.196	53.332	0.013	0.764	0.189
Run 23	40.444	0.009	0.960	0.237	39.012	0.036	0.603	0.210	39.959	0.038	0.562	0.208
Run 24	52.019	0.029	1.280	0.243	48.653	0.028	0.769	0.234	52.449	0.014	0.742	0.217
Run 25	36.028	0.018	0.849	0.187	36.249	0.011	0.542	0.135	36.113	0.047	0.523	0.185
Run 26	48.377	0.012	1.134	0.217	40.890	0.035	0.673	0.170	48.305	0.006	0.678	0.197
Run 27	44.441	0.021	1.059	0.169	37.296	0.048	0.642	0.107	45.289	0.011	0.654	0.175
Run 28	15.865	0.054	0.415	0.204	11.975	0.076	0.331	0.044	14.267	0.042	0.640	0.205
Run 29	13.107	0.018	0.425	0.202	11.024	0.047	0.305	0.179	13.065	0.010	0.259	0.211
Run 30	36.169	0.021	0.910	0.194	35.265	0.032	0.647	0.125	37.575	0.027	0.567	0.211

	PSO				GSAVHO			
Run	N_{fe}	SD	CT	BC	N_{fe}	SD	CT	BC
Run1	32.254	0.030	0.574	0.226	26.381	0.018	0.546	0.015
Run 2	16.335	0.132	0.328	0.240	11.470	0.030	0.300	0.058
Run 3	65.279	0.015	1.013	0.240	46.304	0.010	0.810	0.071
Run 4	47.904	0.009	0.819	0.181	38.942	0.002	0.703	0.065
Run 5	42.021	0.021	0.673	0.242	33.575	0.001	0.625	0.059
Run 6	64.780	0.002	0.899	0.199	48.284	0.015	0.749	0.058
Run 7	18.350	0.012	0.362	0.222	14.179	0.009	0.300	0.070
Run 8	60.479	0.016	0.895	0.229	48.626	0.013	0.745	0.088
Run 9	64.627	0.025	0.909	0.221	50.296	0.004	0.863	0.021
Run 10	43.547	0.009	0.670	0.207	31.526	0.021	0.634	0.063
Run 11	28.981	0.449	0.557	0.195	19.739	0.027	0.508	0.056
Run 12	54.390	0.006	0.809	0.207	45.011	0.010	0.635	0.039
Run 13	21.735	0.020	0.399	0.209	17.516	0.005	0.360	0.046
Run 14	56.060	0.017	0.800	0.297	40.231	0.002	0.735	0.029
Run 15	36.835	0.056	0.767	0.209	25.849	0.034	0.565	0.021
Run 16	23.123	0.007	0.425	0.225	15.660	0.046	0.372	0.071
Run 17	20.028	0.007	0.368	0.209	15.981	0.011	0.330	0.023
Run 18	32.651	0.028	0.531	0.226	23.019	0.035	0.506	0.017
Run 19	16.063	0.105	0.258	0.198	10.831	0.028	0.240	0.070
Run 20	24.019	0.098	0.295	0.191	13.575	0.031	0.257	0.065

Table 2 continued

Run	PSO				GSAVHO			
	N_{fe}	SD	CT	BC	N_{fe}	SD	CT	BC
Run 21	11.873	0.014	0.243	0.236	8.723	0.029	0.227	0.019
Run 22	54.859	0.022	0.924	0.212	45.421	0.006	0.762	0.073
Run 23	41.651	0.032	0.626	0.223	33.317	0.021	0.552	0.019
Run 24	57.209	0.029	0.794	0.266	40.687	0.023	0.743	0.034
Run 25	36.239	0.027	0.578	0.189	25.866	0.005	0.513	0.070
Run 26	48.496	0.017	0.718	0.201	39.691	0.005	0.558	0.033
Run 27	46.607	0.011	0.771	0.216	35.593	0.007	0.454	0.068
Run 28	13.979	0.065	0.299	0.199	18.520	0.069	0.264	0.016
Run 29	13.116	0.026	0.276	0.229	10.168	0.013	0.258	0.029
Run 30	38.664	0.057	0.605	0.189	30.605	0.011	0.565	0.068

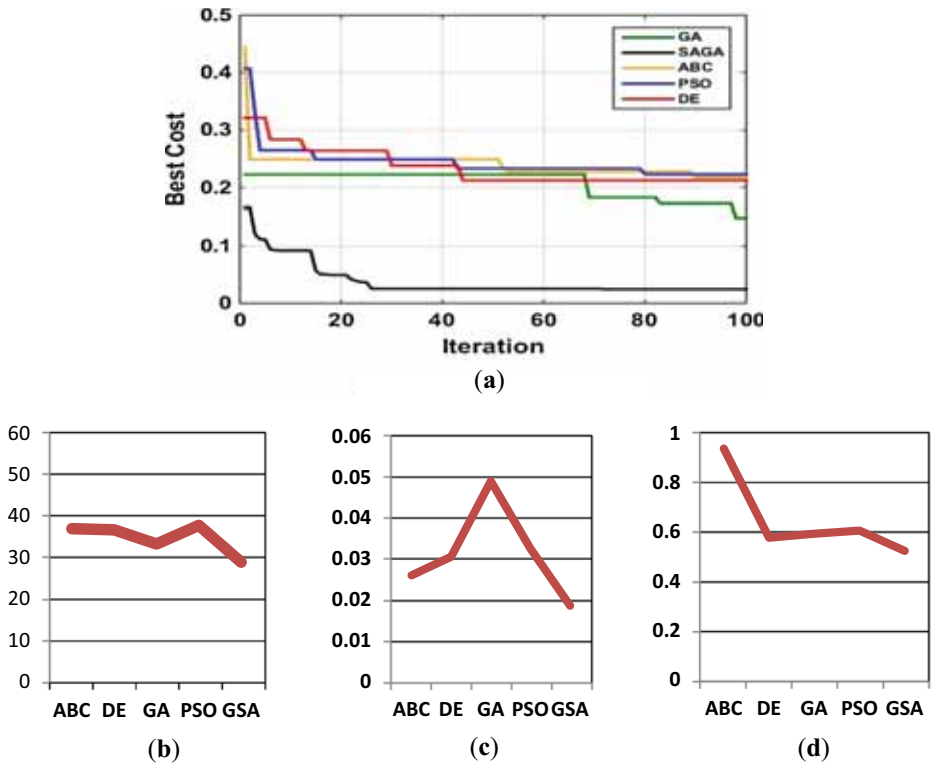


Figure 4. The results obtained by ABC, GA, DE, PSO algorithms versus and GSAVHO algorithm on 30 runs. (**a**) Average of best cost on 30 runs, (**b**) average of N_{fe}, (**c**) Average of SD, and (**d**) Average of CT (Based on second).

more stable solution quality. Table 2 presents the results obtained from ABC, GA, DE, PSO, and GSAVHO algorithms based on N_{fe}, Number of function evaluation; SD, standard deviation; CT, computation time, and BC, best cost. The corresponding graph is given in figure 4. This figure shows the results in 30 different running times with Iteration = 100 and Number of Networks = 3.

The complexity is relative to the number of items, the number of generations and the computation time per generation. Given the usual choices (point mutation, one point crossover, cost function), a genetic algorithm's complexity is O [O (Cost function) × O (mutation) + O (Cost function) × O (crossover)]. The complexity depends on the genetic operators, their implementation (which may have a very significant effect on overall complexity), the representation of the individuals and the population, and obviously on the fitness function. The major computational costs in both GAVHO and GSAVHO are involved in their

mutation and crossover and cost function. Note that, the size of mutation and crossover could be very large; the computational cost of mutation and crossover in GAVHO is much higher than that of mutation and crossover in GSAVHO. This is because in our hybrid algorithm, the T_0 parameter can balance the exploration and exploitation and this matter can reduce the complexity of our heuristic. In addition, it can improve the local search via helping to find good solutions more quickly as it increases the "greediness" within the algorithm. Moreover, the SA rule used in the local search may guide the algorithm towards certain regions of the search space. Therefore, we can conclude that each pass of mutation and crossover in GSAVHO involves less computational cost than mutation and crossover in GAVHO. The updating step needs O (m) comparisons and assignments and the updating of neighboring solutions performs O (Sm) basic operations since its major costs are to compute the values for S solutions. Therefore, the computational complexity of choosing the best solutions in the GSAVHO is O (SmN) since it has N passes and also, the computational complexity of GAVHO is O (mN^2) [32]. If both GSAVHO and GAVHO use the same population size, the ratio between their computational complexities at each generation is: $O(\mathrm{SmN})/O(mN^2)$.

Since S is smaller than N, the variant of GSAVHO has lower computational complexity than GAVHO at each generation.

To avoid attributing the optimization results to the choice of a particular initial population and to conduct fair comparisons, we performed each test at 500 iterations, starting from various randomly selected points in the search domain. The algorithm is coded in MATLAB R (2014b) and the simulations are run on an Intel Core i3 3.3 GHz processor with 8 GB memory capacity.

4. System model

As discussed in section 2.1, the cost function can measure the quality of each network to establish the connections. This metric is able to give the level of merit of each available network in a fixed time. This measurement is computed from a set of weights assigned to each QoS parameter by the AHP method [21], where this allocation can be based on the user's preferences and network conditions. This calculation is shown by the cost function (C), which is computed in the VH decision phase.

We have designed an algorithm based on the hybridization of genetic (GAs) and simulating annealing (SA) algorithms called the genetic simulated annealing vertical handover (GSAVHO). In our algorithm, the individuals of a population are vectors of QoS weights, which are possible solutions of the optimization problem. The population is initially generated with an even number of individuals. Each individual is identified by $\mathrm{pop_{ij}}$ where i is the index matching with the number of individuals in the population and j is an index that corresponds with a QoS parameter. In addition, each individual has a set of cost values, because there are different available networks. The initial population is obtained using random selection. This way, the GSAVHO can start from an initial population where the individuals represent all the areas of the space of solutions, allowing a high degree of diversity for the successive evolutions of the population. The GSAVHO operation is shown in Algorithm 1. Starting from a determined data set, user's profile, number of networks and QoS parameters, precision level, population size, initial temperature ($T_0 = 10$), temperature, reduction rate (alpha = 0.99), number of parents (offsprings) (nc = 2 × round (pc × nPop/2)), number of mutants (nm = round (pm × nPop)), number of iterations (Max It = 1000), number of sub-iterations of SA (MaxSubIt = 10) (line #1) and in (line #2), to avoiding the "Ping-Pong" effect, we need to know whether the handover is necessary. For this purpose, we focus on the SUM of the received signal strength in n networks in a time period T_s based on Eq. (10).

$$SUM = \sum_{T=0}^{T=s} R_T, \tag{10}$$

where s is the number of times that the received signal strength is higher than the trigger threshold of the handover R in a time period; $0 \le T \le s$;

$$\begin{cases} R_n < SUM/n, & \text{no handoff} \\ R_n \ge SUM/n, & \text{execute handover} \end{cases}. \tag{11}$$

By using the above handover triggering and after checking R_n for n_{th} network and comparing the average RSS of networks in the time period s, the 'Ping-Pong' effect can be prevented. At the end of this calculation, the amount of SUM must be reset. The population is initialized (line #3), to continue, crossover percentage (pc) and mutation percentage (pm) are fixed at this stage (line #4), after sorting the population, (line #13). We have defined the pre-decision method to filter the requested services of the users before sending to the GA unit. B_i and R_i are used for the pre-decision stage where they are the available bandwidth (B) and received signal strength (R) of the candidate networks. They are calculated by the following equations.

$$D_{\mathrm{Bandwidth}} = B_2 - B_1 \tag{12}$$

$$D_{\mathrm{RSS}} = R_2 - R_1, \tag{13}$$

where $D_{\mathrm{Bandwidth}}$ is the diversity of available bandwidth between networks and D_{RSS} is the diversity of received signal strength between the networks. Based on following conditions:

$$\begin{cases} \text{if } D_{\mathrm{Bandwidth}} > 0 \text{ and } D_{\mathrm{RSS}} > 0 & \text{Import QoS parameters to decision unit} \\ \text{if } D_{\mathrm{Bandwidth}} \le 0 \text{ and } D_{\mathrm{RSS}} \le 0 & \text{No handof} \end{cases}. \tag{14}$$

In continuation, (line #18 till end), we show the related stages of our optimization algorithm that includes the crossover, mutation, SA rules, and so on.

Algorithm 1 GSAVHO pseudo-code.

Parameters Definition: Initial Temperature, Temp. Reduction Rate, NN, RSS, Set QoS parameters, NQ, nm= Number of Mutants, nPop=Population Size, BT = Bandwidth threshold, RT = RSS threshold, …

```
Calculate the Sum of RSS if R_n ≥ SUM/n continue; else  no handover
Initialize Population
For each node n_i in the network:
Set strategy and percentage for mutation and crossover
Evaluate the cost function of each individual, where:
Calculate priority vector to show relative weights among them by AHP
If B_i > B_T  and R_i > R_T
Calculate D Bandwidth and D_RSS:
If D_Bandwidth > 0 and D_RSS > 0
Import the parameters into the GA
Else
No handover  happens
Sort Population
Initialize Temperature
// GA Main Loop
For it=1: MaxIt
// Perform Crossover
    For subit=1: MaxSubIt
        Select First Parent (w_1) according to C values
        Select Second Parent (w_2) according to C values
        Perform Crossover
        Evaluate Offspring
      End (MaxSubIt)
// Perform Mutation
    For i=1:nm
        Select Parent (w_i) according to C values
        Perform Mutation
        Evaluate Mutants
    End (nm)
Merge Offspring's Population
Sort NEWPOP
Compare using SA Rule
For i=1: nPop
    if new pop(i).Cost<=pop(i).Cost
        pop (i)=new pop(i);
     else
        DELTA=(new pop(i).Cost-pop(i).Cost)/pop(i).Cost;
        P=exp(-DELTA/T);
        if rand<=P
            pop (i)=new pop(i);
    End (if)
Update Best Solution Ever Found
        End (nPop)
        End (MaxIt)
Store Best Cost
Temperature. Reduction
Update Best Solution Ever Found
```

In order to better understanding, we showed the main codes of our algorithm in "Appendix 5".

The following flowchart presents the main steps of our heuristic (figure 5).

5. Computational experiments

5.1 *Evaluation of proposed VHO scheme*

To demonstrate the performance of the VHO optimization in heterogeneous wireless networks, we considered two main experiments: with GSAVHO optimization and with GAVHO optimization. This section focuses on the efficiency of GA–SA as tested against the GA algorithm with different QoS parameters. The networks characterized by the values of the following QoS parameters, are listed in Table 3. In this table, we introduce the QoS parameters of the proposed network selection algorithm. The heterogeneous environment may consist of a number of different wireless networks. For simplicity, without loss of generality, we consider a heterogeneous wireless network consisting of UMTS, Wi-Fi (802.11n), WiMax (802.16), and Wi-Fi (802.11ac).

Selection metrics of cellular networks in the heterogeneous environment are discussed as follows. Handover delay refers to the duration between the initiation and completion of the handover process. Handover delay is related to the complexity of the VHDA process, and reduction of the handover delay is especially important for delay-sensitive voice or multimedia sessions. If D_N is

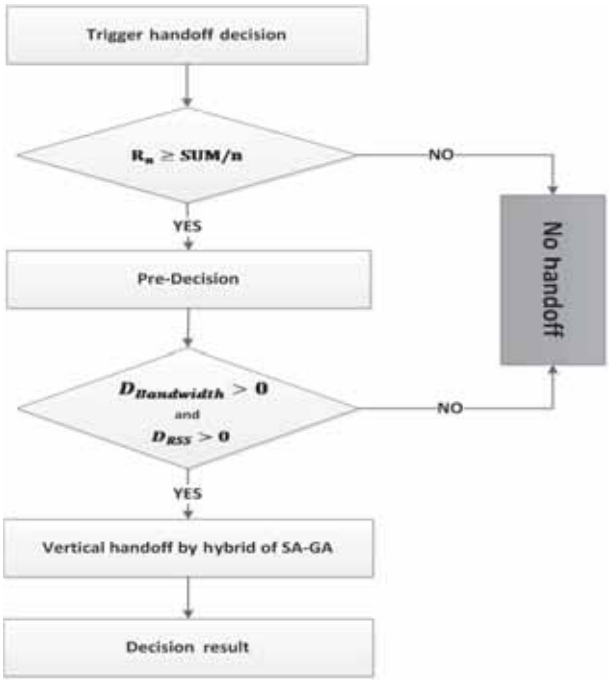


Figure 5. Flowchart of (GSAVHO).

considered as the normalized handoff delay in handoff transition region then for the real-time service, the normalized handoff delay is as follows [33]:

$$D_N = {}^{N\Delta}/_{T_t}, \tag{15}$$

where Δ is the handoff completion time and T_t is the region extending from the point at which the power falls below the threshold for the first time to the point at which the power falls below the threshold permanently.

For delay-tolerant applications, approximate value of the residual bandwidth of the WLAN is evaluated by the following formula (16) [34]:

$$\begin{aligned} \textit{Residual bandwidth} = {} & \textit{throughput} \\ & \times (1 - \alpha \times \textit{channel_utilization}) \\ & \times (1 - \textit{packet_loss_rate}), \end{aligned} \tag{16}$$

where throughput is the throughput that can be shared among mobile terminals in the WLAN, channel utilization is the percentage of time that the access point senses the medium is busy using the carrier sense mechanism, a is a factor that reflects IEEE 802.11 MAC overhead (it is set to 1.25 in Lee *et al* [35]), and packet loss rate is the portion of the transmitted medium access control (MAC) protocol data units (MPDUs) that require retransmission, or are discarded as undeliverable. The values of channel utilization and packet loss rate are obtained from the information in the beacon frame carrying the QoS basic service set (QBSS) load sent by an access point, as defined in IEEE 802.11e [36].

Another metric for network selection algorithm is bit error rate. BER is the probability of bit error rate of cellular network present in heterogeneous environment. Probability of bit error rate is calculated by using Eq. (17), which depends upon signal to noise. Probability of bit error rate defined in Eq. (16) is for a non-coherent FSK in a white Gaussian noise [37].

$$BER_i(\gamma_i) = \frac{1}{2} e^{\gamma_i/2} \tag{17}$$

where γ_i is the signal to noise ratio, $i = 1, 2, 3$ and 4 for UMTS, Wi-Fi (802.11n), WiMax (802.16) and Wi-Fi (802.11ac) cell environments, respectively.

Based on the obtained results, we can see that in GAVHO, in the first iterations such as the iteration between 2 and 15, network 3 is selected as the best solution, and then network 1 is selected as the best network. These values show a level of the network quality by means of cost function. Figures 6 and 7 show results based on different QoS parameters, number of population equals 8, and iteration of 100.

Several experiments were performed to find the best wireless network optimized with the hybridization of GA and SA. The main idea was to achieve a comparison of the results obtained with the GAVHO optimization and the hybridization of the GA–SA optimization for the best network selection during the vertical handover. As mentioned before, each QoS factor includes bandwidth, BER, delay, security level, and monetary cost within the own weight effect on network or user. The values of the QoS parameters characterize a given network and each parameter is combined with its own weight; each weight was obtained using the AHP method. We selected the most main parameters for vertical handover; we then defined and simulated a basic scenario and finally by analyzing the selected parameters, measured and calculated RSS, ABR,

Table 3. QoS parameters.

Net	Bandwidth (Mbps)	BER (dB)	Delay (ms)	Security level	Monetary cost (eur/MB)
UMTS	14.4	0.001	19	8	0.9
Wi-Fi(802.11n)	30	0.01	20	2	0.2
WiMax (802.16)	75	0.01	30	3	0.3
Wi-Fi(802.11ac)	72	0.01	15	3	0.2

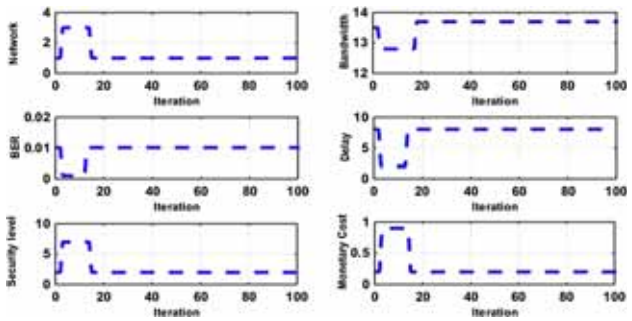


Figure 6. Results of different parameters found by of GAVHO with iteration = 100 and Network's Number = 8.

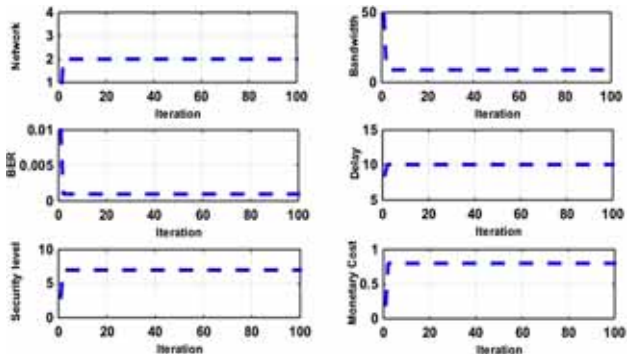


Figure 7. Results of different parameters found by of GSAVHO with iteration = 100 and Network's Number = 8.

SNR, and throughput. We will explain the details of the scenario in the following.

On the GSAVHO algorithm, we have performed simulations to show the feasibility of the proposed scheme using the simulation software QualNet 7.0. Our experiment is for the case of a vertical handover between a 3G network and WLANs. The scenario simulated in QualNet composed of an UMTS BS and IEEE 802.11 APs. The WLAN is based on the IEEE 802.11b standard, and the physical data rate is 11 Mbps. The capacity of a 3G Universal Mobile Telecommunications System is 384 kbps. All links except wireless links each have a capacity of 100 Mbps. Ad-hoc On-demand Distance Vector (AODV) protocol was used as a reactive routing protocol [38]. This protocol offers quick convergence when the ad hoc network topology changes (typically, when a node moves in the network).

As shown in figure 8, the mobile node can be at a given time in the coverage area of an UMTS. However, due to mobility, it can move into the regions covered by more than one access network, i.e., simultaneously within the coverage areas of, for example, an UMTS BS and an IEEE 802.11 AP. Multiple IEEE 802.11 WLAN coverage areas are usually contained within an UMTS coverage area. Since the WLAN1 has a lower coverage range, when the mobile user is moving out of a WLAN1 area, the existence of an accurate and timely handoff decision to maintain the connectivity before the loss of the WLAN access is necessary. Subsequently, the user moves into the regions covered by a UMTS network and then the user would like to move into a WLAN2 area because the user wants to achieve a higher QoS at the lowest cost. Thus, the user changes the connection to the WLAN2. The mobile node associated with UMTS or WLANs monitors and measures the $D_{Bandwidth,}$ which is the diversity of the available bandwidth between networks, and the D_{RSS}, which is the diversity of the received signal strength between the networks of nearby WLANs/UMTS, to check whether an access network with a high data rate is offered.

The performance of the proposed algorithm has been assessed in a scenario when the Mobile Node moves with a constant speed along a straight line path from the area covered by WLAN1 to the one covered by UMTS then

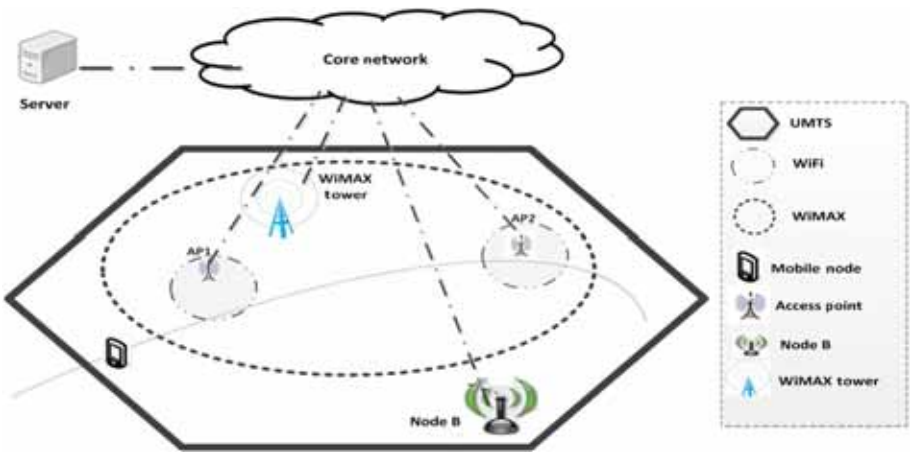


Figure 8. Simulation scenario.

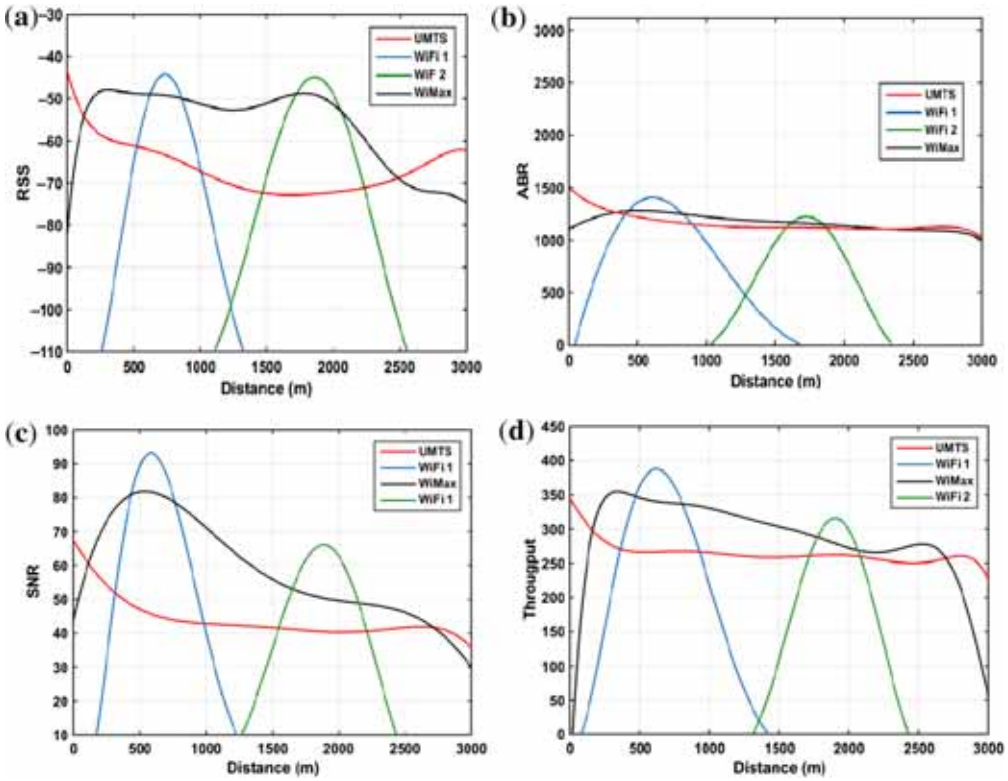


Figure 9. (**a**) Average RSS versus distance, (**b**) available bit rate (ABR) versus distance, and (**c**) SNR versus distance and throughput versus distance in cellular heterogeneous environment of WLANs and UMTS.

roams to the area which is covered by WLAN2. Clearly, with the increase of distance, the average of RSS, ABR, and SNR will be reduced and the BER will be increased. By calculating the cost value based on the number of iterations, the optimal network can be selected as presented in figure 9.

Where γi is the signal to noise ratio, $i = 1$, 2 and 3 for WLANs and UMTS (3G) cell environments, respectively. Bit error rate (BER): For non-coherent FSK in a white Gaussian noise channel, the probability of a bit error is given by: $\mathrm{BER}_i(\gamma i) = \frac{1}{2}e^{\gamma i/2}$, [37]. Available Bit Rate (ABR): $\mathrm{ABR}_i =$ bandwidth $\times \log_2(1 + \mathrm{SNR}_i)$ $i = 1$, 2, 3, 4 [39]. Signal to Noise Ratio (SNR): $\mathrm{SNR}_i = \frac{E_\mathrm{b}}{N_\mathrm{o}}$ [37], E_b is the received energy per bit and is noise power of the channel.

5.2 *Comparison with SEFISA and FMADM*

We have performed performance comparisons between our algorithm GSAVHO and other algorithms structured in the literature, namely SEFISA, GAVH [3], and FMADM [40]. In the study by Jaraiz-Simon *et al* [3], two algorithms were designed to decide on the best network to establish connection in a vertical handover process; SEFISA is based on the simulated annealing (SA) algorithm and GAVH was developed based on genetic algorithms (GAs). In addition, the FMADM [40] is a fuzzy multiple attribute decision making algorithm that selects a suitable wireless access network during the vertical handover process.

We selected these proposals because this study enabled us to compare our heuristic with similar datasets. In order to make fair comparisons, our heuristics have five similar QoS parameters in dataset1 and nine QoS parameters in dataset2. We have considered the following datasets for the weight ranges of QoS parameters, which are shown in Tables 4 and 5. The datasets consist of several networks characterized by the following QoS parameters:

Table 4. Simulation results corresponding to the best costs for GAVH, SEFISA, GA and GSAVHO for number of networks = 4, number of QoS = 5 (dataset 1).

	B	E	D	S	C
NN = 1	0.44	0.02	0.5	0.02	0.02
NN = 2	0.54	0.36	0.02	0.07	0.01
NN = 3	0.48	0.50	0.04	0.02	0.02
NN = 4	0.34	0.68	0.02	0.04	0.02
Best cost					
SEFISA			1.89		
GAVH			5.77		
GSAVHO			4.7248e–6		
Standard deviation					
SEFISA			N/A		
GAVH			N/A		
GSAVHO			0.0430		

Table 5. Simulation results corresponding to the best costs for FMADM and GSAVHO for number of networks = 3 and number of QoS = 9 (dataset 2).

	N	L	T	B	W	V	S	G	C
NN = 1	1.82	0.54	2.46	1.10	0.44	2.46	2.46	2.46	0.54
NN = 2	0.04	0.09	0.04	0.45	0.04	0.04	0.07	0.04	0.21
NN = 3	0.46	0.02	0.02	0.46	0.02	0.02	0.02	0.14	0.28
Best cost									
FMADM					0.799				
GSAVHO					4.6861e–06				
Standard deviation									
FMADM					N/A				
GSAVHO					0.0044				

B = bandwidth (kbps), E = BER (dB), D = delay (ms), S = (dB), C = cost (eur/MB), L = network latency (ms), J = jitter (ms), R = burst error, A = average retransmissions/packet, P = packet loss (%), G = received signal strength indication RSSI (dBm), N = network coverage area (km), T = reliability, W = battery power requirement (W), and V = mobile terminal velocity (m/s). Among these algorithms, the one based on the hybridization of GA and SA (GSAVHO) demonstrated the best performance, in terms of precision and cost function.

6. Conclusion

Our goal was to achieve an optimized GSAVHO with a small cost function, and for this reason, the SA rules were not changed and we proposed an optimization for GA where only the combination and comparison among populations of parents and children were based on the simulated annealing rules. The proposed approach performed well in several test problems in terms of the number of cost function evaluations required, the quality of the solutions found, the average of the function evaluation numbers, and the average of cost functions. Several experiments were performed for the optimization of the vertical handover decision-making algorithms with GA and the hybridization of GA–SA in heterogeneous wireless networks. The results obtained were analyzed and compared, and it was found that the GSAVHO had a lower cost function compared to the GAVHO. Simulation results indicated that our proposed VHDA was able to minimize the cost function, reduce the number of unnecessary handovers, avoid the 'Ping-Pong' effect, and select the best access network that was optimized to network conditions, quality of service requirements, mobile terminal conditions, user preferences, and service cost.

Appendices

Appendix 1: Artificial bee colony (ABC)

```
clc;
clear;
close all;
%% Problem Definition
 CostFunction=@(x) Sphere(x);        % Cost Function
 nVar=5;              % Number of Decision Variables
 VarSize=[1 nVar];    % Decision Variables Matrix Size
 VarMin=-10;          % Decision Variables Lower Bound
VarMax= 10;          % Decision Variables Upper Bound
%% ABC Settings
 MaxIt=500;              % Maximum Number of Iterations
 nPop=200;               % Population Size (Colony Size)
 nOnlooker=nPop;         % Number of Onlooker Bees
 L=round(0.5*nVar*nPop); % Abandonment Limit Parameter
 a=1;                    % Acceleration Coefficient Upper Bound
%% Initialization
% Empty Bee Structure
empty_bee.Position=[];
empty_bee.Cost=[];
% Initialize Population Array
pop=repmat(empty_bee,nPop,1);
% Initialize Best Solution Ever Found
BestSol.Cost=inf;
% Create Initial Population
for i=1:nPop
    pop(i).Position=unifrnd(VarMin,VarMax,VarSize);
    pop(i).Cost=CostFunction(pop(i).Position);
      if pop(i).Cost<=BestSol.Cost
        BestSol=pop(i);
    end
end
% Abandonment Counter
C=zeros(nPop,1);
% Array to Hold Best Cost Values
BestCost=zeros(MaxIt,1);
%% ABC Main Loop
 for it=1:MaxIt
% Recruited Bees
    for i=1:nPop
% Choose k randomly, not equal to i
        K=[1:i-1 i+1:nPop];
        k=K(randi([1 numel(K)]));
% Define Acceleration Coeff.
        phi=unifrnd(-a,+a,VarSize);
% New Bee Position
        newbee.Position=pop(i).Position+phi.*(pop(i).Position-
pop(k).Position);
% Evaluation
        newbee.Cost=CostFunction(newbee.Position);
% Comparision
```

```
          if newbee.Cost<=pop(i).Cost
              pop(i)=newbee;
          else
              C(i)=C(i)+1;
          end
          end
  % Calculate Fitness Values and Selection Probabilities
      F=zeros(nPop,1);
      for i=1:nPop
          if pop(i).Cost>=0
              F(i)=1/(1+pop(i).Cost);
          else
              F(i)=1+abs(pop(i).Cost);
          end
      end
      P=F/sum(F);
  % Onlooker Bees
      for m=1:nOnlooker
  % Select Source Site
          i=RouletteWheelSelection(P);
  % Choose k randomly, not equal to i
          K=[1:i-1 i+1:nPop];
          k=K(randi([1 numel(K)]));
  % Define Acceleration Coeff.
          phi=unifrnd(-a,+a,VarSize);
  % New Bee Position
          newbee.Position=pop(i).Position+phi.*(pop(i).Position-
pop(k).Position);
  % Evaluation
          newbee.Cost=CostFunction(newbee.Position);
  % Comparision
          if newbee.Cost<=pop(i).Cost
              pop(i)=newbee;
          else
              C(i)=C(i)+1;
          end
      end

  % Scout Bees
      for i=1:nPop
          if C(i)>=L
              pop(i).Position=unifrnd(VarMin,VarMax,VarSize);
              pop(i).Cost=CostFunction(pop(i).Position);
              C(i)=0;
          end
      end
  % Update Best Solution Ever Found
      for i=1:nPop
          if pop(i).Cost<=BestSol.Cost
              BestSol=pop(i);
         end
   end
  % Store Best Cost Ever Found
   BestCost(it)=BestSol.Cost;
  % Display Iteration Information
   disp(['Iteration    '    num2str(it)    ':    Best    Cost    =    '
num2str(BestCost(it))]);
   end
```

Appendix 2: Genetic algorithm (GAVHO)

```
  clc;
   clear;
   close all;
   %% Problem Definition
   CostFunction=@Sphere;        % Cost Function
   nVar=10;                     % Number of Variables
   VarSize=[1 nVar];            % Size of Variables Matrix
   VarMin=-5;                   % Lower Bound of Variables
   VarMax= 5;                   % Upper Bound of Variables
   VarRange=[VarMin VarMax];    % Variation Range of Variables
   VelMax=(VarMax-VarMin)/10;   % Maximum Velocity
   VelMin=-VelMax;              % Minimum Velocity
   %% GA Parameters
   MaxIt=1000;      % Maximum Number of Iterations
   nPop=50;         % Population Size
   pCrossover=0.7;                          % Crossover Percentage
   nCrossover=round(pCrossover*nPop/2)*2;    % Number of Parents
 (Offsprings)
   pMutation=0.2;                       % Mutation Percentage
   nMutation=round(pMutation*nPop);     % Number of Mutants
   %% Initialization
   % Empty Structure to Hold Individuals Data
   empty_individual.Position=[];
   empty_individual.Cost=[];
   % Create Population Matrix
   pop=repmat(empty_individual,nPop,1);
   % Initialize Positions
   for i=1:nPop
      pop(i).Position=unifrnd(VarMin,VarMax,VarSize);
      pop(i).Cost=CostFunction(pop(i).Position);
  end
   % Sort Population
   pop=SortPopulation(pop);
   % Store Best Solution
   BestSol=pop(1);
   % Vector to Hold Best Cost Values
   BestCost=zeros(MaxIt,1);
   %% GA Main Loop
    for it=1:MaxIt
   % Crossover
     popc=repmat(empty_individual,nCrossover/2,2);
     for k=1:nCrossover/2
         i1=randi([1 nPop]);
         i2=randi([1 nPop]);
         p1=pop(i1);
         p2=pop(i2);
         [popc(k,1).Position
popc(k,2).Position]=Crossover(p1.Position,p2.Position,VarRange);

popc(k,1).Cost=CostFunction(popc(k,1).Position);
         popc(k,2).Cost=CostFunction(popc(k,2).Position);
             end
```

```
    popc=popc(:);
% Mutation
    popm=repmat(empty_individual,nMutation,1);
    for k=1:nMutation
                i=randi([1 nPop]);
                p=pop(i);
                popm(k).Position=Mutate(p.Position,VarRange);
                popm(k).Cost=CostFunction(popm(k).Position);
            end
% Merge Population
    pop=[pop
         popc
         popm];
% Sort Population
    pop=SortPopulation(pop);
% Delete Extra Individuals
    pop=pop(1:nPop);
% Update Best Solution
    BestSol=pop(1);
% Store Best Cost
    BestCost(it)=BestSol.Cost;
% Show Iteration Information
    disp(['Iteration ' num2str(it) ': Best Cost = '
num2str(BestCost(it))]);
    end
```

Appendix 3: Differential evolution (DE)

```
clc;
clear;
close all;
%% Problem Definition
 CostFunction=@(x) Sphere(x);    % Cost Function
 nVar=20;             % Number of Decision Variables
 VarSize=[1 nVar];    % Decision Variables Matrix Size
 VarMin=-5;           % Lower Bound of Decision Variables
 VarMax= 5;           % Upper Bound of Decision Variables
%% DE Parameters
 MaxIt=1000;      % Maximum Number of Iterations
 nPop=50;         % Population Size
 beta_min=0.2;    % Lower Bound of Scaling Factor
 beta_max=0.8;    % Upper Bound of Scaling Factor
 pCR=0.2;         % Crossover Probability
%% Initialization
 empty_individual.Position=[];
 empty_individual.Cost=[];
 BestSol.Cost=inf;
 pop=repmat(empty_individual,nPop,1);
 for i=1:nPop
 pop(i).Position=unifrnd(VarMin,VarMax,VarSize);
 pop(i).Cost=CostFunction(pop(i).Position);
        if pop(i).Cost<BestSol.Cost
        BestSol=pop(i);
    end
    end
```

```
   BestCost=zeros(MaxIt,1);
  %% DE Main Loop
   for it=1:MaxIt
          for i=1:nPop
                  x=pop(i).Position;
                  A=randperm(nPop);
                  A(A==i)=[];
                  a=A(1);
                  b=A(2);
                  c=A(3);
  % Mutation
  %beta=unifrnd(beta_min,beta_max);
          beta=unifrnd(beta_min,beta_max,VarSize);
          y=pop(a).Position+beta.*(pop(b).Position-
pop(c).Position);
  % Crossover
   z=zeros(size(x));
   j0=randi([1 numel(x)]);
   for j=1:numel(x)
       if j==j0 || rand<=pCR
           z(j)=y(j);
       else
           z(j)=x(j);
       end
   end

                  NewSol.Position=z;
          NewSol.Cost=CostFunction(NewSol.Position);
                  if NewSol.Cost<pop(i).Cost
              pop(i)=NewSol;
                          if pop(i).Cost<BestSol.Cost
                 BestSol=pop(i);
              end
          end
              end
  % Update Best Cost
      BestCost(it)=BestSol.Cost;
  % Show Iteration Information
      disp(['Iteration   '   num2str(it)   ':   Best   Cost   =   '
num2str(BestCost(it))]);
      end
```

Appendix 4: Particle swarm optimization (PSO)

```
  clc;
  clear;
  close all;
  %% Problem Definition
  global NFE;
  CostFunction=@(x) Sphere(x);        % Cost Function
  nVar=5;              % Number of Decision Variables
  VarSize=[1 nVar];    % Size of Decision Variables Matrix
  VarMin=-10;          % Lower Bound of Variables
  VarMax= 10;          % Upper Bound of Variables
  %% PSO Parameters
  MaxIt=100;       % Maximum Number of Iterations
  nPop=100;         % Population Size (Swarm Size)
```

```
% Constriction Coefficients
phi1=2.05;
phi2=2.05;
phi=phi1+phi2;
chi=2/(phi-2+sqrt(phi^2-4*phi));
w=chi;          % Inertia Weight
wdamp=1;        % Inertia Weight Damping Ratio
c1=chi*phi1;    % Personal Learning Coefficient
c2=chi*phi2;    % Global Learning Coefficient
% Velocity Limits
VelMax=0.1*(VarMax-VarMin);
VelMin=-VelMax;
%% Initialization
empty_particle.Position=[];
empty_particle.Cost=[];
empty_particle.Velocity=[];
empty_particle.Best.Position=[];
empty_particle.Best.Cost=[];
particle=repmat(empty_particle,nPop,1);
GlobalBest.Cost=inf;
for i=1:nPop
% Initialize Position
    particle(i).Position=unifrnd(VarMin,VarMax,VarSize);
% Initialize Velocity
    particle(i).Velocity=zeros(VarSize);
% Evaluation
    particle(i).Cost=CostFunction(particle(i).Position);
% Update Personal Best
    particle(i).Best.Position=particle(i).Position;
    particle(i).Best.Cost=particle(i).Cost;
% Update Global Best
    if particle(i).Best.Cost<GlobalBest.Cost
                GlobalBest=particle(i).Best;
            end
    end
  BestCost=zeros(MaxIt,1);
  nfe=zeros(MaxIt,1);
%% PSO Main Loop
  for it=1:MaxIt
        for i=1:nPop
% Update Velocity
        particle(i).Velocity = w*particle(i).Velocity ...
            +c1*rand(VarSize).*(particle(i).Best.Position-
particle(i).Position) ...
            +c2*rand(VarSize).*(GlobalBest.Position-
particle(i).Position);
% Apply Velocity Limits
        particle(i).Velocity                                    =
max(particle(i).Velocity,VelMin);
        particle(i).Velocity                                    =
min(particle(i).Velocity,VelMax);
% Update Position
        particle(i).Position    =    particle(i).Position    +
particle(i).Velocity;
% Velocity Mirror Effect
        IsOutside=(particle(i).Position<VarMin                  |
particle(i).Position>VarMax);
        particle(i).Velocity(IsOutside)=-
particle(i).Velocity(IsOutside);
% Apply Position Limits
```

```
          particle(i).Position                                    =
max(particle(i).Position,VarMin);
          particle(i).Position                                    =
min(particle(i).Position,VarMax);
  % Evaluation
          particle(i).Cost = CostFunction(particle(i).Position);
  % Update Personal Best
          if particle(i).Cost<particle(i).Best.Cost

particle(i).Best.Position=particle(i).Position;
              particle(i).Best.Cost=particle(i).Cost;
  % Update Global Best
              if particle(i).Best.Cost<GlobalBest.Cost
                                GlobalBest=particle(i).Best;
                            end
                        end
              end
      BestCost(it)=GlobalBest.Cost;
          nfe(it)=NFE;
          disp(['Iteration    '    num2str(it)    ':    NFE    =    '
num2str(nfe(it)) ', Best Cost = '
  num2str(BestCost(it))]);
          w=w*wdamp;
     end
```

Appendix 5: GSAVHO Algorithm

```
function [BestCost nfe AllBestSAGA BestSAGA BandwidthSAGA
BERSAGA DelaySAGA SecurityLevelSAGA
MondetaryCostSAGA]=saga(SelecRandomNet)
CostFunction=@(x) Sphere(x);        % Cost Function
SetNetParameter=@(Net) InitializeNetwork(Net);
nVar=9;             % Number of Decision variables
VarSize=[1 nVar];   % Variables Martix Size
VarMin=-10;         % Variables Lower Bound
VarMax= 10;         % Variables Upper Bound
if numel(VarMin)==1                 % If VarMin is Scalar
    VarMin=repmat(VarMin,VarSize);  % Convert VarMin to Vector
end
if numel(VarMax)==1                 % If VarMax is Scalar
    VarMax=repmat(VarMax,VarSize);  % Convert VarMax to Vector
end
%% GA Parameters
tic;
MaxIt=500;        % Maximum Number of Iterations
MaxSubIt=10;       % Maimum Number of Sub-iterations
T0=10;             % Initial Temp.
alpha=0.99;        % Temp. Reduction Rate
nPop=3;           % Population Size
pc=2;                    % Crossover Percentage
nc=2*round(pc*nPop/2);  % Number fo Parents (Offsprings)
pm=1;                    % Mutation Percentage
nm=round(pm*nPop);       % Number of Mutants
gamma=0.2;               % Crossover Inflation Rate
mu=0.1;                  % Mutation Rate
MutationMode='rand';     % Mutation Mode
eta=0.1;                     % Mutation Step Size Ratio
sigma=eta*(VarMax-VarMin);  % Mutation Step Size
%% Initialization
```

```
global NFE;
NFE=0;
% Create Empty Structure
empty_individual.Position=[];
empty_individual.Cost=[];
empty_individual.Net=[];
AllBestSAGA=[];
BestSAGA.Position=[];
BestSAGA.Cost=[];
BestSAGA.Net=[];
BandwidthSAGA=[];
BERSAGA=[];
DelaySAGA=[];
SecurityLevelSAGA=[];
MondetaryCostSAGA=[];

% Create Structre Array to Save Population Data
pop=repmat(empty_individual,nPop,1);
% Initilize Population
for i=1:nPop
    % Create Random Solution (Position)
     %SelecRandomNet=randi(4);
pop(i).Position=SetNetParameter(SelecRandomNet(i));%unifrnd(VarM
in,VarMax,VarSize);
   % Evaluate Solutions
    pop(i).Cost=CostFunction(pop(i).Position);
     pop(i).Net=SelecRandomNet(i);
end
 %Temporary
temp=pop;
 % Sort Population
Costs=[pop.Cost];
[Costs, SortOrder]=sort(Costs);
pop=pop(SortOrder);
 % Store Best Solution Ever Found
BestSol=pop(1);
 % Create Array to Hold Best Cost Values
BestCost=zeros(MaxIt,1);
 % Create Array to Hold NFEs
nfe=zeros(MaxIt,1);
 % Initialize Temp.
T=T0;
 %% GA Main Loop
 for it=1:MaxIt
 for subit=1:MaxSubIt
         % Perform Crossover
        popc=repmat(empty_individual,nc/2,2);
        for k=1:nc/2
             % Select First Parent
            i1=randi([1 nPop]);
            p1=pop(i1);
             % Select Second Parent
            i2=randi([1 nPop]);
            while i1==i2 i2=randi([1 nPop]); end
            p2=pop(i2);
             % Perform Crossover
            [popc(k,1).Position, popc(k,2).Position]=...

ArithmeticCrossover(p1.Position,p2.Position,gamma,VarMin,VarMax)
;
```

```
            % Evaluate Offsprings
            popc(k,1).Cost=CostFunction(popc(k,1).Position);
            popc(k,2).Cost=CostFunction(popc(k,2).Position);
             %Network
            popc(k,1).Net=p1.Net;
            popc(k,2).Net=p2.Net;
        end
        popc=popc(:);
         % Perform Mutation
        popm=repmat(empty_individual,nm,1);
        for l=1:nm
             % Select Parent
            i=randi([1 nPop]);
            p=pop(i);

            % Perform Mutation

              popm(l).Position=...

Mutate(p.Position,mu,MutationMode,sigma,VarMin,VarMax);
              % Evaluate Mutatnt
              popm(l).Cost=CostFunction(popm(l).Position);
               popm(l).Net=p.Net;
          end
             % Merge Offsprings Population
         newpop=[popc
                 popm];
                 % Sort NEWPOP
         [~, SortOrder]=sort([newpop.Cost]);
         newpop=newpop(SortOrder);
             % Compare using SA Rule
         for i=1:nPop
             if newpop(i).Cost<=pop(i).Cost
                 pop(i)=newpop(i);
                 else
                 DELTA=(newpop(i).Cost-pop(i).Cost)/pop(i).Cost;
                 P=exp(-DELTA/T);
                 if rand<=P
                     pop(i)=newpop(i);
                 end
             end
             % Update Best Solution Ever Found
             if pop(i).Cost<=BestSol.Cost
                 BestSol=pop(i);
             end
                end
             end
         % Store Best Cost
    BestCost(it)=BestSol.Cost;
         %***************************
    BestSAGA(it).Position=BestSol.Position;
    BandwidthSAGA(it)=temp(BestSol.Net).Position(1);
    BERSAGA(it)=temp(BestSol.Net).Position(2);
    DelaySAGA(it)=temp(BestSol.Net).Position(3);
    SecurityLevelSAGA(it)=temp(BestSol.Net).Position(4);
    MondetaryCostSAGA(it)=temp(BestSol.Net).Position(5);
    BestSAGA(it).Cost=BestSol.Cost;
    BestSAGA(it).Net=BestSol.Net;
    AllBestSAGA(it)=BestSol.Net;
    %***************************
```

```
    % Store NFE
    nfe(it)=NFE;
    % Show Iteration Information
    disp(['Iteration ' num2str(it) ...
          ': Best Cost = ' num2str(BestCost(it)) ...
          ', NFE = ' num2str(nfe(it))]);
    % Temp. Reduction
    T=alpha*T;
    end
    toc;
fprintf('TIC TOC: %g\n', toc);
```

References

[1] Chandralekha Praffula Behera and Behera Praffula Kumar 2010 Minimization of number of handoff using genetic algorithm in heterogeneous wireless networks. *Int. J. Latest Trends Comput.* 1(2): 24–28

[2] Zhang Chengbo, Wang Xingwei and Huang Min 2013 A multi-objective genetic algorithm based handoff decision scheme with ABC supported. *Intelligent computing theories.* pp 217–226 Springer

[3] Jaraiz-Simon M D, Gomez-Pulido J A and Vega-Rodriguez M A 2015 Embedded intelligence for fast QoS-based vertical handoff in heterogeneous wireless access networks. *Pervasive Mobile Comput* 19: 141–155

[4] Lin Feng-Tse, Kao Cheng-Yan and Hsu Ching-Chi 1993 Applying the genetic approach to simulated annealing in solving some NP-hard problems. *IEEE Trans. Syst. Man Cybern.* 23(6): 1752–1767

[5] Li Xun-Gui and Wei Xia 2008 An improved genetic algorithm-simulated annealing hybrid algorithm for the optimization of multiple reservoirs. *Water Resour. Manag.* 22(8): 1031–1049

[6] Giupponi Lorenza, Agusti Ramón, Pérez-Romero Jordi and Sallent Oriol 2005 A novel joint radio resource management approach with reinforcement learning mechanisms. *Paper presented at the 24th IEEE International Performance, Computing, and Communications Conference, 2005. IPCCC 2005*

[7] Wilson A, Lenaghan A and Malyan R 2005 Optimising wireless access network selection to maintain qos in heterogeneous wireless environments. *Paper presented at the Wireless Personal Multimedia Communications*

[8] Ahmed Atiq, Boulahia Leıla Merghem and Gaïti Dominique 2014 Enabling vertical handover decisions in heterogeneous wireless networks: A state-of-the-art and a classification. *IEEE Commun. Surveys Tutorials* 16(2): 776–811

[9] Movahedi Zeinab, Ayari Mouna, Langar Rami and Pujolle Guy 2012 A survey of autonomic network architectures and evaluation criteria. *IEEE Commun. Surveys Tutorials* 14(2): 464–490

[10] Paul Subharthi, Pan Jianli and Jain Raj 2011 Architectures for the future networks and the next generation Internet: A survey. *Comput. Commun.* 34(1): 2–42

[11] TalebiFard Peyman, Wong Terrence and Leung Victor C M 2010 Access and service convergence over the mobile internet – A survey. *Comput. Netw.* 54(4): 545–557

[12] Rakovic Valentin and Gavrilovska Liljana 2010 Novel RAT selection mechanism based on Hopfield neural networks. *Paper presented at 2010 International Congress on Ultra Modern Telecommunications and Control Systems and Workshops (ICUMT)*

[13] Çalhan Ali and Çeken Celal 2013a Artificial neural network based vertical handoff algorithm for reducing handoff latency. *Wireless Personal Commun.* 71(4): 2399–2415

[14] Kordos Mirosław and Duch Włodzisław 2004 Variable step search algorithm for MLP training. *Paper presented at the 8th IASTED International Conference on Artificial Intelligence and Soft Computing*, Marbella, Spain

[15] Lera Gabriel and Pinzolas Miguel 2002 Neighborhood based Levenberg-Marquardt algorithm for neural network training. *IEEE Trans. Neural Networks* 13(5): 1200–1203

[16] ÇAlhan Ali and ÇEken Celal 2013b Case study on handoff strategies for wireless overlay networks. *Comput. Standards Interfaces* 35(1): 170–178

[17] Nan Wang, Wenxiao Shi, Shaoshuai Fan and Shuxiang Liu 2011 PSO-FNN-based vertical handoff decision algorithm in heterogeneous wireless networks. *Proc. Environ. Sci.* 11: 55–62

[18] Liu Xia and Jiang Ling-ge 2012 A novel vertical handoff algorithm based on fuzzy logic in aid of grey prediction theory in wireless heterogeneous networks. *J. Shanghai Jiaotong Univ. (Sci.)* 17: 25–30

[19] Singhrova Anita and Prakash Nupur 2012 Vertical handoff decision algorithm for improved quality of service in heterogeneous wireless networks. *IET Commun.* 6(2): 211–223

[20] Pahlavan Kaveh, Krishnamurthy Prashant, Hatami Ahmad, Ylianttila Mika, Makela J, Pichna Roman and Vallstron Jari 2000 Handoff in hybrid mobile data networks. *IEEE Personal Commun.* 7(2): 34–47

[21] Ahmed T, Kyamakya K, Ludwig M, Anne K R, Schroeder J, Galler S, Kyamakya K and Jobmann K 2006 A context-aware vertical handover decision algorithm for multimode mobile terminals and its performance: na

[22] Saaty Thomas L 1988 *What is the analytic hierarchy process?* Springer

[23] Karaboga Dervis 2005 *An idea based on honey bee swarm for numerical optimization*: Technical report-tr06, Erciyes university, engineering faculty, computer engineering department

[24] Karaboga D, Gorkemli B, Ozturk C and Karaboga N 2014 A comprehensive survey: Artificial bee colony(ABC) algorithm and applications. *Artif. Intell. Rev.* 42(1): 21–57

[25] John Holland 1992 *Holland, Adaptation in natural and artificial systems*. MIT Press, Cambridge, MA

[26] Li Zichuan and Schonfeld Paul 2015 Hybrid simulated annealing and genetic algorithm for optimizing arterial signal timings under oversaturated traffic conditions. *J. Adv. Transp.* 49(1): 153–170

[27] Price Kenneth, Storn Rainer M and Lampinen Jouni A 2006 *Differential evolution: A practical approach to global optimization*. Springer Science & Business Media

[28] AlRashidi Mohammed R and El-Hawary Mohamed E 2009 A survey of particle swarm optimization applications in electric power systems. *IEEE Trans. Evolut. Comput.* 13(4): 913–918

[29] Aarts Emile and Korst Jan 1988 *Simulated annealing and Boltzmann machines*.

[30] Tsai Ching-Chih, Huang Hsu-Chih and Chan Cheng-Kai 2011 Parallel elite genetic algorithm and its application to global path planning for autonomous robot navigation. *IEEE Trans. Ind. Electron.* 58(10): 4813–4821

[31] Dede Tayfun and Ayvaz Yusuf 2015 Combined size and shape optimization of structures with a new meta-heuristic algorithm. *Appl. Soft Comput.* 28: 250–258

[32] Deb K, Pratap A, Agarwal S and Meyarivan T A M T 2002 A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Trans. Evolut. Comput.* 6(2): 182–197

[33] Park Hyo Soon, Yoon Sung Hoon, Kim Tae Hyoun, Park Jung Shin, Do Mi Sun and Lee Jai Yong 2003 Vertical handoff procedure and algorithm between IEEE802. 11 WLAN and CDMA cellular network. *Mobile Commun.* pp 103–112: Springer

[34] Yan Xiaohuan, Şekercioğlu Y Ahmet and Narayanan Sathya 2010 A survey of vertical handover decision algorithms in Fourth Generation heterogeneous wireless networks. *Comput. Netw.* 54(11): 1848–1863

[35] Lee Cheng Wei, Chen Li Ming, Chen Meng Chang, and Sun Yeali Sunny 2005 A framework of handoffs in wireless overlay networks based on mobile IPv6. *IEEE J. Select. Areas Commun.* 23(11): 2118–2128

[36] Mangold S, Choi S, Hiertz G R, Klein O and Walke B 2003 Analysis of IEEE 802.11 e for QoS support in wireless LANs. *IEEE Wireless Commun.* 10(6): 40–50

[37] Reza Fazlay Rabby 2012 Optimum ranges for data transmission in mobile communications. *Int. J. Sci. Eng. Res.* 3: 481–489

[38] Perkins Charles E and Royer Elizabeth M 1999 Ad-hoc on-demand distance vector routing. *Paper presented at the Second IEEE Workshop on Mobile Computing Systems and Applications, 1999*. Proceedings. WMCSA'99

[39] Pfister Henry D, Soriaga Joseph B and Siegel Paul H 2001 On the achievable information rates of finite state ISI channels. *Paper presented at the Global Telecommunications Conference, 2001. GLOBECOM'01. IEEE*

[40] Nkansah-Gyekye Yaw and Agbinya Johnson I 2007 Vertical handoff decision algorithm for UMTS-WLAN. *Paper presented at the 2nd International Conference on Wireless Broadband and Ultra Wideband Communications, 2007*. AusWireless 2007

[41] Rakesh Jha and Dalal Upena 2010 *A survey of mobile WiMax IEEE 802.16 m standard*. arXiv preprint arXiv:1005.0976

[42] Shaddad R Q, Mohammad A B, Al-Gailani S A, Al-hetar A M and Elmagzoub M A 2014 A survey on access technologies for broadband optical and wireless networks. *J. Netw. Comput. Appl.* 41: 459–472

[43] Singh A K and Mishra B 2012 Comparative study on wireless local area network standards. *Int. J. Appl. Eng. Technol.* 2(3): 1–4

[44] Banerji S and Chowdhury R S 2013 *On IEEE 802.11: Wireless LAN technology*. arXiv preprint arXiv:1307.2661